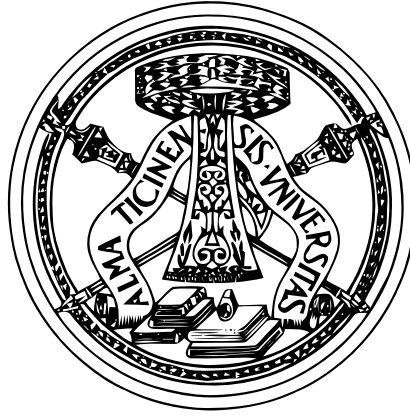




Corso di laurea triennale in Artificial Intelligence

REPORT OF THE FINAL EXAM PROJECT – PLANISUSS

Computer Programming, Algorithms and Data Structures

Module 1

Author: Villa Giulia

ID: 514193

Academic Year 2024 / 2025

INDEX

CHAPTER 1 – INTRODUCTION	3
1.1 Project Goals and Motivation	3
1.2 Overview of the Planisuss World	3
CHAPTER 2 – REQUIREMENTS, SPECIFICATIONS AND CONSTRAINTS	5
2.1 Requirements and Specifications	5
2.2 Constraints and Assumptions	6
CHAPTER 3 – MODULES IMPLEMENTATION DETAILS	8
3.1 Constants	8
3.2 World Representation	9
3.3 Species	10
3.3.1 Vegetob	10
3.3.2 Animal	10
3.3.3 Erbast	11
3.3.4 Carviz	12
3.4 Social Groups	13
3.4.1 Herd	13
3.4.2 Pride	14
3.5 Simulation Control	15
3.5.1 GameManager	15
3.6 Main and Plotting	16
3.6.1 Main	16
3.6.2 User Interface	18
CHAPTER 4 – TESTING AND EVALUATION	20
4.1 Simulation Scenarios and Parameters	20
4.2 Results and Visualization Output	21
4.3 Performance Observations	23
CHAPTER 5 – LIMITATIONS AND FUTURE WORK	25
5.1 Limitations, Improvements, Extensibility Considerations	25
CHAPTER 6 – CONCLUSION	27
TOOLS AND RESOURCES	28
REFERENCES	29

CHAPTER 1 – INTRODUCTION

This chapter introduces the **Planisuss** project, a simulation of an artificial ecosystem designed to explore species interaction, environmental dynamics and spatial population modelling within a grid-based world. **Planisuss** introduces multiple species with distinct roles, probabilistic behaviour, energy-based lifecycles and social grouping mechanisms that co-exist in the ecosystem.

The content of this chapter establishes the conceptual foundations of the project. It begins by outlining the primary objectives and motivation of the simulation, followed by a high-level overview of the world structure and the core ecological mechanisms.

1.1 Project Goals and Motivation

The primary objective of this project is the design and implementation of a simulation framework that models the dynamics of an artificial ecosystem called **Planisuss**. Inspired by classical cellular automata such as **Wa-Tor** and **Conway's Game of Life**, the simulation seeks to explore emergent behaviours arising from interaction rules among multiple species, within a grid-based environment.

The simulated world consists of three types of entities: **Vegetob**, **Erbast** and **Carviz**, each characterized by distinct biological roles and behavioural rules. **Vegetob**, a plant species, serves as a renewable food source for **Erbasts**, a herbivore species. In turn, **Erbasts** are preyed upon by **Carvizes**, carnivorous predators. The spatial and temporal evolution of these species is governed by mechanisms such as movement, feeding, reproduction and intra- or inter-species conflict.

Beyond merely encoding these ecological rules, the project also emphasizes data visualization as a central feature. One of the core motivations is to create a simulation that is not only functional, but also interpretable through real-time visual feedback. To this end, the simulation is coupled with interactive visualizations built using the **matplotlib** library. These visualizations allow users to observe population distributions, trajectories and trends over time, thereby offering a qualitative and quantitative window into the ecosystem's evolution.

A secondary, but equally important, motivation lies in exploring the design of **object-oriented** architectures suitable for simulations with non-trivial entity interactions. **Encapsulation** of behaviour within species classes and modular code structure are goals that guided the software design.

1.2 Overview of the Planisuss World

The **Planisuss** simulation models a closed ecosystem composed of interdependent species operating within a finite, two-dimensional grid. The world is represented as a rectangular matrix of cells of size **NUMCELLS_R** x **NUMCELLS_C**. Each **Cell** is assigned a terrain type, either **ground** or **water**, at the beginning of the simulation. All border cells are initialized as **water**, forming a natural boundary that animals cannot cross, while inner cells are probabilistically assigned based on the user-configurable **WATER_RATIO** parameter. Only **ground** cells are capable of supporting life, acting as the substrate for vegetative growth and animal activity.

Each cell may contain a local instance of vegetation (**Vegetob**), zero or more individual animals and potentially social groups: **Herd**s of **Erbasts** or **Pride**s of **Carvizes**. These inhabitants interact with one another and with the environment through a discrete-time simulation process structured around daily cycles.

- **Vegetob** is a non-animal species that grows in **ground** cells. Its growth rate is dynamically adjusted based on its current density, accelerating when the density is low and slowing as it approaches the maximum value. If all neighbouring **ground** cells surrounding a given cell reach full **Vegetob** density, the **Vegetob** in that cell is said to "overwhelm" it, which results in the elimination of all animal life present.
- **Erbasts**, the herbivorous species, consume **Vegetob** to replenish energy. Each individual is characterized by parameters such as energy level, age, lifetime and a social attitude factor ranging from 0 (completely individualistic) to 1 (fully cooperative). **Erbasts** may either act independently or form **Herd**s to coordinate behaviour. Their movement is driven by localized evaluation of neighbouring cells, factoring in **Vegetob** availability, predator presence and previously visited locations.
- **Carvizz**, the carnivorous species, prey upon **Erbasts** and are similarly defined by energy, age, lifetime and social behaviour metrics. Their ability to hunt successfully depends not only on individual characteristics such as hunting efficiency and base success rate, but also on the coordination level within their **Pride**. If a group of **Carvizz**s fails to find prey or consistently loses hunting encounters, both their social cohesion and effectiveness degrade over time. On the other hand, successful hunts improve their energy reserves and sharpen their hunting traits.

Social organization plays a key role in both species' dynamics. When multiple individuals of the same species are present in a cell, they may form a group, either a **Herd** (of **Erbasts**) or a **Pride** (of **Carvizz**s), provided with capacity constraints (**MAX_HERD** and **MAX_PRIDE** respectively). Groups coordinate collective decisions such as movement and resource consumption, reducing the computational overhead of treating each individual separately. However, individual members may choose to split from the group depending on their energy and social attitude. **Prides** may also engage in intra-species conflict when more than one exists in the same cell. In such cases, either a merge is negotiated based on group cohesion, or a series of one-on-one last blood fights occurs until only one **Pride** remains.

The simulation advances in units called **days**, each of which is composed of five distinct **phases**:

- During the **growth phase**, **Vegetob** densities are increased in all **ground** cells.
- In the **movement phase**, individuals and social groups assess neighbouring **ground** cells within a radius defined by the **NEIGHBORHOOD** parameter and move accordingly.
- In the third phase, either **grazing** or **hunting phase**, **Erbasts** that remained stationary graze on **Vegetob**, while **Carvizz**s engage in hunting behaviour if prey is available.
- The **procreation phase** increases each animal's age and initiates reproduction for individuals that reach the end of their lifespan. **Offsprings** inherit attributes from their parent according to predefined rules: energy is split, and other attributes such as lifetime, social attitude and species-specific attributes are passed down with controlled variation.
- In the **global event phase**, **migration** or **epidemic** events may be triggered depending on current population levels and configured thresholds. **Migrations** introduce new individuals if a species' population falls dangerously low, while **epidemics** drastically reduce population if it exceeds sustainable bounds.

The **Planisuss** simulation is modular by design. All species, behaviours and world mechanics are encapsulated in dedicated classes, and the simulation itself is coordinated by a centralized **GameManager**. Simulation constants are loaded from a configuration file and can be modified by the user through a graphical **User Interface** prior to launch. This structure allows for flexibility and extensibility, making the system well-suited for both experimentation and future feature development.

CHAPTER 2 – REQUIREMENTS, SPECIFICATIONS AND CONSTRAINTS

This chapter defines the formal requirements and operational rules that govern the structure and behaviour of the **Planisuss** simulation. It outlines the specifications that each component of the system must fulfil, from world generation and species attributes to simulation flow and event handling. The system is expected to accurately reflect interactions between species and their environment, while remaining configurable and computationally efficient.

In addition to the explicit rules derived from the project brief, the chapter presents several implementation-specific constraints and modelling assumptions. These include decisions made to ensure ecological plausibility, simplify complex behaviour, or resolve architectural limitations. Together, the specifications and constraints described here provide the logical and technical framework within which the simulation has been developed and tested.

2.1 Requirements and Specifications

As defined in the provided project's specifications, the **Planisuss** simulation must fulfil several functional and non-functional requirements to meet the objectives defined in the project brief. At its core, the system is required to simulate a self-contained ecosystem composed of plant life and two interacting animal species, each governed by distinct biological, behavioural and social rules. The simulation must be implemented using **Python**, **NumPy** and **matplotlib**.

The world must be represented as a two-dimensional grid, with user-configurable dimensions (**NUMCELLS_R**, **NUMCELLS_C**). Each **Cell** in the grid must be designated as either **ground** or **water**, with border cells always initialized as **water** and inner cells randomly assigned based on a **WATER_RATIO**. Only **ground** cells may contain living entities or support vegetation.

The simulation environment must accommodate three species:

- **Vegetob**: a non-animal species responsible for providing energy to herbivores. It must grow automatically at each time step, with a growth rate that decreases as the local density approaches the upper limit.
- **Erbast**: a herbivorous species characterized by energy, age, lifetime and social attitude. **Erbasts** consume **Vegetob**, may move between cells and can form **Herds** to coordinate behaviour. Each individual must act according to local cell evaluation logic and energy constraints.
- **Carviz**: a carnivorous species that hunts **Erbast**. **Carvizes** must also track energy, age, lifetime and social metrics. They can form **Prides**, engage in coordinated hunts and fight with rival **Prides** over territory.

Each **day** in the simulation is structured into five **phases**: **Vegetob growth**, animal **movement**, **grazing** or **hunting**, **procreation** and **global events**. All species must follow rules defined for energy expenditure, lifespan progression, reproduction and death. **Offsprings** must inherit traits from their parent according to specified logic, including energy splitting and attribute averaging.

The simulation must incorporate social group dynamics, such that individuals in the same cell may form or join a **Herd** or **Pride**. Groups must coordinate movement and resource usage, and individuals may choose to split from the group based on their internal state. **Prides** must be able to evaluate whether to merge or fight when multiple groups occupy the same space. **Herds** must exhibit collision resolution when more than one group is present in a single cell.

The proposed solution of the simulation allows for automatic and conditional events, including:

- **Migrations**, triggered when the population of a species drops below a given fraction of the expected count.
- **Epidemics**, triggered when the population exceeds a configurable threshold, resulting in large-scale mortality.

Visualization is a core requirement. The simulation must render:

- A real-time map view of the world, color-coded by species density and terrain.
- Time series plots tracking population counts and **Vegetob** density.
- Interfaces to pause, resume, and save the simulation, and optionally inspect simulation statistics and current constants.

The software must include a graphical configuration interface where users can adjust key parameters prior to simulation startup, including world size, population ratios and group capacity limits. The final implementation must maintain a modular structure, enabling future extensions to species behaviour, ecosystem complexity and UI components. The system must also support state persistence via **save** and **load** functionality using the **pickle module**.

2.2 Constraints and Assumptions

The design and implementation of the **Planisuss** simulation are governed by a set of logical constraints and modelling assumptions intended to maintain ecological consistency, computational tractability and adherence to project requirements.

A primary constraint is the spatial limitation imposed by the grid structure. The world is finite and bounded: animals cannot cross **water** cells or the outer perimeter of the map, and all interactions are limited to a predefined neighbourhood radius (**NEIGHBORHOOD**). Only **ground** cells are viable for vegetation and animal life, and each day's decisions must be made using strictly local information within the animal's neighbourhood.

A key modelling assumption is that only vertical and horizontal neighbours are considered, both for movement and neighbourhood evaluation. Diagonal adjacency is explicitly excluded. The simulation uses a **Manhattan distance** model to define the neighbourhood: two cells are neighbours if the sum of their row and column offsets is greater than zero and less than or equal to the specified radius. This assumption applies uniformly across all species, movement routines, and interaction rules.

In the **growth phase**, **Vegetob** grows dynamically in each **ground** cell. A critical assumption here is that a cell is considered "overwhelmed" by **Vegetob** (and its animals are terminated) if all of its neighbouring **ground** cells have reached maximum **Vegetob** density. **Water** cells in the neighbourhood are ignored in this evaluation.

Regarding social group formation, different rules apply to **Herds** and **Prides**. **Herds** of **Erbasts** are allowed to coexist within the same cell if each has reached its **MAX_HERD** capacity, and no merging is mandated in such cases, but only one **Herd** can stay in the current cell, so the others are forced to move. In contrast, for **Prides** of **Carvizz**, coexistence in a single cell is not permitted beyond a single group. If multiple **Prides** are present, they must either merge, provided their combined size does not exceed **MAX_PRIDE**, or engage in a conflict sequence, eliminating one another until a single **Pride** remains.

An additional modelling assumption concerns predation behaviour. Both individual **Carvizz** and entire **Prides** are limited to hunting a single **Erbast** per day. This assumption ensures that the predator-prey

interaction remains bounded and prevents cascading eradication of prey populations within a single time step. When a **Pride** hunts, it coordinates the attack but still targets only one **Erbast**, the one with the lowest energy in the cell. Likewise, individual **Carvizz** seek out a single prey.

Energy management introduces several implicit constraints. **Movement costs** are constant and must be paid upfront; animals lacking sufficient energy are effectively immobilized. Animals also lose energy monthly due to aging (**AGING_E**, **AGING_C**). **Vegetob** consumption is competitive: when the available density is insufficient to feed all **Erbasts** in a **Herd**, energy is distributed to the weakest individuals. **Carvizz** gain energy only if the hunt is successful; failed hunts result in energy loss and trait degradation.

The simulation includes global control mechanisms to maintain ecological balance. **Migrations** occur when a species' population drops below a set threshold, introducing new individuals, while **epidemics** reduce overgrown populations by randomly eliminating individuals at a fixed death rate. Both events are limited by cooldown periods (**MIGRATION_COOLDOWN_DAYS**, **EPIDEMIC_COOLDOWN_DAYS**) to prevent excessive frequency of such events.

Finally, visualization and persistence introduce operational constraints. The simulation must remain responsive to user controls, render updates in real time and support **saving** and **reloading** full simulation state without data loss. To meet these conditions, the simulation engine is time-stepped in discrete units and designed to execute each day's events atomically.

CHAPTER 3 – MODULES IMPLEMENTATION DETAILS

This chapter provides a detailed account of the architecture and internal logic of the **modules** that compose the **Planisuss** simulation system. Each component is implemented in a dedicated **Python** file and designed according to **object-oriented programming principles**, with an emphasis on **modularity**, **encapsulation** and **reusability**. The goal of this section is to explain how the various entities interact, how their behaviour is structured and how the overall simulation is coordinated through successive phases.

- **Note 1:** A portion of the system's configurability is exposed to the user through a graphical interface prior to simulation startup. In particular, the values of selected constants can be set interactively, with each input validated against defined constraints. The interface also displays the default values and admissible ranges, ensuring both flexibility and consistency in configuration.
- **Note 2:** Several **modules** include the directive from `__future__` **import annotations**. This was necessary to address issues arising from **cyclic imports**, which occur when two or more classes refer to each other in their type annotations. Without this mechanism, **Python** evaluates type hints at definition time, making such circular references problematic. By enabling **lazy evaluation** of annotations (i.e., treating them as strings to be resolved at runtime), this import allows for complete and type-safe class definitions without causing import errors or requiring structural compromises in the code.
- **Note 3:** All classes implement explicit **getter** and **setter** methods for their attributes. This design choice supports the principle of **encapsulation**, ensuring that any external interaction with an object's internal state occurs through well-defined methods. While these accessors do not currently return **defensive copies** or systematically **validate input** parameters, given the self-contained nature of the project, they provide a clear structure that can be extended to enforce stricter integrity checks in the event of future integration with external components. Within the class itself, direct attribute access remains permissible, as it does not compromise **encapsulation**.

3.1 Constants

The **constants.py** file defines the set of global parameters that govern the structural and behavioural configuration of the **Planisuss** simulation. This file serves as a centralized reference for all tunable values that influence species characteristics, environmental conditions, event triggers and User Interface behaviour.

Constants in this module are grouped logically by function: world dimensions, initial population ratios, **Vegetob** growth behaviour, animal aging and energy constraints, social group limits and global control events such as migrations and epidemics. The structure of the file, supported by inline comments and clear naming conventions, promotes readability and simplifies the tuning of simulation parameters.

Several constants defined in this module are exposed to the user through the configuration interface prior to launching the simulation. This allows users to define **custom scenarios** while ensuring all input values remain within valid ranges. The application performs **input validation** at runtime to ensure that user-provided values comply with model requirements.

Throughout the system, the use of constants rather than hardcoded values, facilitates consistency and flexibility. All behavioural logic in the simulation (e.g., movement cost, aging, maximum lifespan, group

size limits) references these values explicitly, ensuring that changes to configuration propagate correctly and predictably across all **modules**.

For a complete list of constants and their associated values and roles, refer directly to the source code in the **constants.py** file.

3.2 World Representation

The **Cell** class represents the fundamental spatial unit of the **Planisuss** world. Each cell corresponds to a single position in the simulation grid and encapsulates all environmental and biological information relevant to that location. The class is defined in the **cell.py** module and serves as a central interface for interactions between entities and the environment.

Upon initialization, each **Cell** is assigned a fixed terrain type, either **ground** or **water**, based on its location in the grid and the simulation's water distribution ratio. **Ground** cells are capable of supporting life and are initialized with a **Vegetob** instance whose density is randomly drawn within a predefined range. **Water** cells, by contrast, are uninhabitable.

The **Cell** class maintains internal lists for all possible inhabitants:

- **Erbast individuals** and **Carviz individuals** are tracked separately until they are grouped.
- **Herds** and **Prides** represent social groupings of **Erbasts** and **Carvizes**, respectively, and are maintained in their own lists after group formation.

To support **encapsulation** and **modular interaction**, the class provides explicit **getter** and **setter** methods for all attributes. Additionally, methods are defined to compute local statistics, such as the total number of **Erbasts** or **Carviz** in the cell, and to retrieve the weakest **Erbast** among individuals or **Herds**.

A key component of the class is its ability to manage social group formation and reorganization. Each **day**, the method **initiate_daily_social_groups()** is called to perform the following sequence:

1. Remove invalid groups (empty or single-member).
2. Form new **Herds** and **Prides** from any ungrouped individuals.
3. Merge existing **Herds** when possible.
4. Either merge or initiate combat between competing **Prides** occupying the same cell.

This mechanism ensures that social groups are dynamically reconstructed each **day** based on current population and spatial distribution, aligning with the simulation's rules for group behaviour and territorial conflict.

The class also implements logic for **Vegetob** interactions. It includes methods to **grow** or **consume** **Vegetob**, and to assess whether a cell has been "overwhelmed". The latter occurs when all neighbouring **ground** cells reach maximum **Vegetob** density, leading to the death of all animals in the current cell. Importantly, **water** cells are excluded from this evaluation, in accordance with the assumptions defined in the simulation rules.

Additional utility methods support common operations such as adding or removing individuals and groups, resetting cell state, or enforcing game mechanics like elimination due to overwhelming **Vegetob**. All such methods interact seamlessly with the rest of the system through the use of proper type annotations and method-level **encapsulation**.

3.3 Species

The simulation models three distinct species: **Vegetob**, **Erbast** and **Carviz**, each with its own behavioural logic, attributes and role. While **Vegetob** represents passive plant life, **Erbast** and **Carviz** are active animal species capable of movement, feeding, reproduction and social behaviour. The animal classes share a common base through the **Animal** superclass, which defines the core attributes and behavioural traits. The following sections describe the implementation details of each species.

3.3.1 Vegetob

Vegetob is the plant life of the **Planisuss** ecosystem and is the only source of food for the **Erbast** population. It is implemented by the **Vegetob** class, which is instantiated once for each **ground** cell during initialization and is controlled and managed directly by the respective **Cell** object.

The primary property of the **Vegetob** class is its **density**, which is a floating-point value between **MIN_VEGETOB_DENSITY** and **MAX_VEGETOB_DENSITY**. This value corresponds to the nutrients in the cell and changes over time according to two primary processes: growth and consumption.

The **grow()** function models daily **Vegetob** growth. Instead of increasing by a fixed constant, **growth()** is implemented as an active process: the rate of growth is proportional to remaining capacity (i.e., current density minus the limit). This makes growth happen faster in cells where **Vegetob** density is low and slow down as **Vegetob** density approaches full capacity.

Conversely, the **consume()** method is used while in the grazing stage, where **Erbasts** or **Herds** request a specific amount of **Vegetob** from a given cell. The method suitably reduces the internal density and then returns the exact amount consumed, which could be less than that requested in the case of insufficient available **Vegetob**.

Vegetob plays a central role in establishing dynamics in the ecosystem. In fact, **Vegetob**'s presence influences the movement decision of **Erbast** populations and **Herd** group. Additionally, when **Vegetob** reaches maximum density in all the neighbouring **ground** cells, it causes a critical state that results in the extinction of animals in the corresponding cell, thus making **Vegetob** not only a source of nutrition but also a potential ecological threat if not kept under control.

3.3.2 Animal

The **Animal** class defines the common characteristics and behavioural model for all animal species in the **Planisuss** simulation. It serves as an abstract superclass for both **Erbast** and **Carviz**, **encapsulating** shared attributes such as energy, age, lifetime, social behaviour and spatial positioning. This design provides a uniform foundation for species-specific **extensions**, while avoiding redundancy in code structure.

Each **Animal** instance maintains a reference to its current **Cell** and an **age** that increases with each simulation **day**. Animals also possess a **max_lifetime** parameter which determines their lifespan and a Boolean flag **alive** that tracks whether the individual remains in the simulation. Once either energy or age limits are exceeded, the animal is considered dead and is removed from the system. If death occurs due to aging, the animal is eligible to reproduce, creating two **offsprings** according to inheritance rules defined in the **reproduce()** method.

The class also includes a **social_attitude** attribute, which influences whether the animal will follow its social group or act independently. Animals with higher social attitude are more likely to participate in collective movement, while those with low values tend to behave autonomously. Each animal keeps a **queue** of previously visited cells, used to avoid repetitive movement and support better decision-making.

Movement is modelled explicitly through the **move_to()** method. An animal may relocate to a neighbouring **ground** cell if it is alive, has sufficient energy and the target cell lies within the Manhattan-distance neighbourhood defined by **NEIGHBORHOOD**. Movement incurs a fixed energy cost and is restricted to one action per **day**. Movement history is updated after each relocation.

The **age_one_day()** method models daily biological progression. It increases the animal's age, applies monthly energy loss (**AGING_E** or **AGING_C** depending on species) and checks for mortality due to energy depletion or lifespan expiration. In the latter case, the animal invokes **reproduce()**, which creates two **offsprings** that inherit its traits. Energy is divided between them, and other numerical attributes are split with random asymmetry while preserving consistency with the parent's values.

Social group membership is managed through the **current_group** attribute. When animals join or leave a **Herd** or **Pride**, this reference is updated accordingly. Group capacity is checked before reproduction: animals are allowed to spawn **offsprings** only if the resulting group does not exceed its defined maximum size (**MAX_HERD** and **MAX_Pride** depending on the type of group).

The class also implements behavioural decision-making. The **can_follow_group()** method combines social attitude and relative energy to determine whether an animal should adhere to its group's movement or split off.

3.3.3 Erbast

The **Erbast** class models the herbivorous species in the **Planisuss** ecosystem and is implemented as a subclass of **Animal**. It inherits all core characteristics and behavioural traits defined in the superclass while introducing species-specific attributes and methods relevant to grazing, movement evaluation and reproduction.

Each **Erbast** is initialized with randomized values for energy and lifetime, drawn from the intervals defined by **INITIAL_ENERGY_MIN_E**, **INITIAL_ENERGY_MAX_E**, **MIN_LIFE_E** and **MAX_LIFE_E**. In addition to the attributes inherited from **Animal**, every **Erbast** maintains a **preferred_vegetob_density** value, representing the ideal **Vegetob** concentration the individual seeks when evaluating potential destinations.

Erbasts consume **Vegetob** through the **graze()** method. This method is invoked either by the individual or collectively within a **Herd** when the animal chooses to remain stationary during the movement phase. Grazing increases the **Erbast**'s energy, subject to the maximum threshold defined by **MAX_ENERGY_E**, and reduces the **Vegetob** density of the local cell. When **Vegetob** is insufficient to feed all members of a group, energy is allocated to those with the lowest energy.

Movement decisions for individual **Erbasts** are based on a desirability function defined in **evaluate_cell_desirability_erbast()**. The function assigns a score to each neighbouring **ground** cell by combining several weighted components:

- The difference between the local **Vegetob** density and the individual's preferred level.
- The number of other **Erbasts** present, scaled by social attitude.
- A penalty for the presence of **Carviz** predators.

- A minor penalty for revisiting recently visited cells.

The **evaluate_movement()** method compares the desirability of the current cell with all neighbouring options and returns the destination with the highest score. If no better cell is found, the **Erbast** remains stationary and graze.

During the procreation phase, if an **Erbast** dies of old age and the reproduction constraints are met, two **offsprings** are generated through the inherited **reproduce()** method. The **apply_inheritance_rules()** method is **overridden** to include transmission of the **preferred_vegetob_density** attribute. The **offsprings** are added either to the parent's **Herd** or as independent individuals in the cell, depending on the parent's group membership at the time of death.

Each **Erbast** also maintains a reference to its current **Herd**, allowing coordinated movement and group-level decision-making when applicable.

3.3.4 Carviz

The **Carviz** class models the carnivorous species in the **Planisuss** simulation. As a subclass of **Animal**, it inherits the fundamental characteristics and methods associated with energy, aging, movement and social affiliation, while extending the model with species-specific properties related to predation and group hunting behaviour.

Each **Carviz** is initialized with randomized values for initial energy and lifespan within the bounds defined by **INITIAL_ENERGY_MIN_C**, **INITIAL_ENERGY_MAX_C**, **MIN_LIFE_C** and **MAX_LIFE_C**. In addition to the inherited traits, a **Carviz** maintains three specific attributes:

- **hunting_efficiency**, which scales the effective strength of the individual in hunting interactions.
- **hunting_success_rate**, representing a probabilistic baseline for successful attacks.
- **days_since_last_hunt**, a counter that influences behaviour and performance over time.

Predatory behaviour is implemented through the **hunt()** method, which takes a target **Erbast** and a coordination bonus provided by the **Carviz's Pride**, if applicable. The success of a hunt depends on the attacker's energy, its efficiency, its success rate and the coordination factor. If the attack is successful, the **Erbast** is killed and its energy is absorbed by the **Carviz**. Otherwise, the **Carviz** loses energy and suffers a reduction in its hunting-related attributes. Only a single **Erbast** may be hunted per **day** by each **Carviz** or **Pride**, consistent with the simulation's ecological constraints.

The method **search_erbast_to_hunt()** defines how prey is selected. The search prioritizes individual **Erbasts** in the current cell; if none are present, the method falls back to searching within **Herds**. The selected target is always the individual with the lowest energy, supporting both ecological realism and game balance.

Movement decisions for **Carviz** are implemented using a desirability function defined in **evaluate_cell_desirability_carviz()**. The evaluation includes:

- Social attraction, proportional to the number of other **Carvizes** in the cell and scaled by social attitude.
- Presence of **Erbasts**, which positively influences desirability.
- A "desperation" bonus if the **Carviz** has not hunted in several days.

The method **evaluate_movement()** selects the most desirable cell from the local neighbourhood and triggers relocation if a better option exists. If the **Carviz** remains in place and prey is found, it attempts to hunt.

Reproduction occurs only upon death by aging and is governed by the same constraints applied to other **Erbasts**: **offsprings** are generated only if the resulting **Pride** does not exceed the configured maximum size (**MAX_PRIDE**). The **apply_inheritance_rules()** method is **overridden** to incorporate **Carviz**-specific attributes. Both hunting efficiency and success rate are passed down to the **offsprings** with proportional variation, allowing the population to evolve across generations.

Carviz may operate individually or as part of a **Pride**. Group membership affects both movement decisions and hunting dynamics. Within **Prides**, **Carviz** coordinate hunting actions through a shared **hunting_coordination** value derived from the aggregate traits of their members. Members who split from the group due to low energy or social disposition act independently and may attempt solo hunts.

3.4 Social Groups

In the **Planisuss** simulation, animal species are capable of forming social groups that enable coordinated behaviour. These groupings are modelled through two distinct classes: **Herds** for **Erbasts** and **Prides** for **Carvizes**. Social groups influence decision-making processes such as movement, feeding and reproduction, while also affect the internal dynamics of each species. The following sections describe the logic and implementation details of these collective entities.

3.4.1 Herd

The **Herd** class models the collective behaviour of **Erbast** individuals that share the same cell and choose to coordinate their actions. It provides mechanisms for group formation, collective movement and dynamic restructuring based on membership changes. A **Herd** is created when two or more ungrouped **Erbasts** occupy the same cell and are eligible for aggregation under the constraints defined by **MAX_HERD**.

Upon instantiation, a **Herd** is initialized with a reference to its host **Cell** and a list of member **Erbast** objects. Each member is removed from the cell's individual **Erbast** list and assigned the current **Herd** as its **social_group**. The **Herd** maintains its own Boolean movement flag (**can_move**), a Boolean flag (**forced_to_move**) indicating whether it is forced to move due to the presence of other **Herds** in the same cell, and a list of recently visited cells used in group-level movement decisions, implemented as a **queue**.

The method **collective_gaze()** implements the group behaviour. If the total number of members exceeds the local **Vegetob** density, energy is allocated to members with the lowest energy, otherwise all members graze. The flag **can_move** is disabled for the remainder of the **day** once grazing occurs.

Movement is handled collectively through **decide_collective_movement()** and **collective_move_to()**. The group evaluates a list of available **ground** cells in its neighbourhood. Each member scores each cell using their individual desirability function (**evaluate_cell_desirability_erbast()**), and the final score is computed as the average across the group. The **Herd** moves to the highest-scoring cell, which can also be the current cell and stay in it if that is the case. During the relocation, individual members incur energy loss and their movement history is updated. If a member no longer has sufficient energy or is not inclined to follow the group, it is removed from the **Herd** and behaves independently.

Membership in the **Herd** is dynamic. The method **add_member()** adds a new **Erbast** to the group, provided it does not exceed capacity. Social attitude is increased upon successful inclusion. Conversely, **remove_member()** removes an **Erbast** from the group and restores it to the cell's list of ungrouped individuals. The class also defines an **eliminate()** method that removes the group entirely if it becomes invalid (i.e., has no members or only one).

A **Herd's** ability to move may be constrained by inter-group competition. When multiple **Herds** coexist in the same cell and none can merge due to capacity limits, only one is allowed to remain stationary, while the others are flagged with **forced_to_move = True** and must relocate during the **movement phase**.

Lastly, the **age_all_members()** method is used during the **procreation phase** to increment the **age** of each member, apply monthly energy penalties and evaluate reproduction conditions. **Offsprings** are automatically added to the same **Herd** unless doing so would violate group size limits.

3.4.2 Pride

The **Pride** class implements the social structure for **Carviz** individuals that occupy the same cell and coordinate their actions. Unlike **Herds**, which may coexist in the same location if group limits are met, multiple **Prides** are not permitted to remain simultaneously in a cell. If more than one **Pride** is present, they must either merge or engage in combat until only one remains. This rule reflects the territorial and aggressive nature of the species and is enforced systematically each **day** through the cell's social group reorganization routine.

A **Pride** is initialized with a list of member **Carviz** instances and a reference to the **Cell** in which it resides. Upon creation, all members are removed from the cell's list of ungrouped **Carvizes** and assigned the current **Pride** as their **current_group**. Each **Pride** maintains an internal list of visited cells, implemented as a **queue**, a Boolean movement flag (**can_move**) and a derived metric known as **hunting_coordination**, which quantifies the group's cooperative hunting capability. This value is computed based on the average social attitude, average hunting efficiency and average hunting success rate of its members, adjusted by a size factor.

The method **collective_hunt()** governs group-level predation behaviour. Each **day**, a **Pride** may attempt to hunt a single **Erbast** in its current cell, targeting first the weakest ungrouped individual and then those within **Herds** if necessary. The attack is carried out by the member with the highest energy. If the hunt succeeds, the attacker gains energy and improves its hunting traits, while the remaining members receive a smaller portion of energy and a modest increase in skill. If the hunt fails or no prey is found, all members incur energy loss and a decline in social attitude and hunting attributes.

Movement is coordinated through **decide_collective_movement()** and **collective_move_to()**. The destination is chosen based on the average desirability of each available cell, as evaluated by members using their individual desirability function (**evaluate_cell_desirability_carviz()**). A bonus is applied to cells containing **Erbasts**, scaled by the group's **hunting_coordination** value. During relocation, each member pays the movement energy cost and updates its **Cell** and group references. Members that are unable or unwilling to follow the group are removed and treated as individuals, allowing them to still attempt a solitary hunt before the end of the day.

Group membership is adjusted dynamically through **add_member()** and **remove_member()** methods, both of which update the member's attributes and recalculate the group's coordination metric. The **eliminate()** method dissolves the **Pride** if it becomes invalid due to insufficient membership.

Conflict resolution between **Prides** is handled at the cell level. When two or more **Prides** occupy the same cell, they first evaluate whether to merge, based on the average social attitude of their members and the maximum group size. If merging is not mutually acceptable or would exceed **MAX_PRIDE**, a series of one-on-one last blood fights is initiated. Champion members with the highest energy duel until one group is eliminated. Survivors receive a social attitude bonus proportional to the configured **SOCIAL_BOOST_FROM_FIGHT**.

Lastly, the **age_all_members()** method is used during the **procreation phase** to increment the **age** of each member, apply monthly energy penalties and evaluate reproduction conditions. **Offsprings** are automatically added to the same **Pride** unless doing so would violate group size limits.

3.5 Simulation Control

Simulation control in **Planisuss** is managed through a dedicated **GameManager** class that governs the evolution of the ecosystem over time. This class coordinates daily phases execution, population updates, event triggers and state management. The following section describe the internal structure and responsibilities of the **GameManager** class.

3.5.1 Game Manager

The **GameManager** class serves as the central controller of the simulation lifecycle. It is responsible for initializing the environment, executing the daily update sequence, tracking global state and managing high-level events such as **migrations** and **epidemics**. All temporal progression and coordination across modules pass through this class.

At initialization, the **GameManager** creates the simulation grid as a two-dimensional list of **Cell** objects, using the dimensions defined by **NUMCELLS_R** and **NUMCELLS_C**. Border cells are always set as **water**, while inner cells are probabilistically designated as **ground** or **water** based on the **WATER_RATIO** parameter. For each **ground** cell, a **Vegetob** instance is created with a randomized initial density, and references to these **ground** cells are stored for efficient access.

The method **populate_initial_animals()** determines the initial population counts for **Erbasts** and **Carvizz** based on configurable ratios and places randomly generated individuals into **ground** cells. These animals are stored directly in the cell's individual lists until grouped during the first **day**.

The simulation advances through discrete time steps using the **step()** method. Each step increases the internal simulation **clock** and triggers a full daily cycle via **simulate_one_day()**. A **day** is composed of five **phases**:

1. **Growth**, in which **Vegetob** density increases in all ground cells.
2. **Movement, grazing and hunting**, where animals make movement decisions based on local evaluations and act accordingly.
3. **Procreation**, which increments animal age, applies energy penalties and triggers reproduction for individuals that reach their lifetime.
4. **Migration**, which repopulates the world if either species drops below a configured fraction of its expected size.
5. **Epidemic**, which reduces population if it exceeds a threshold multiple of the expected size.

During the second phase, the method **phases_movement_grazing_hunting()** organizes all individual and group-level actions. Social groups are formed and restructured at the start of the day using each

cell's ***initiate_daily_social_groups()*** method. ***Herds*** and ***Prides*** then make collective movement decisions and their members either move or graze or hunt. Individual animals follow similar logic. Coordination of this logic ensures that each animal acts only once per ***day***, and all flags (***can_move***, ***forced_to_move***) are reset at the end of the phase.

The simulation also tracks long-term metrics for visualization and analysis. Each day, values such as total population, number of social groups and average ***Vegetob*** density are recorded through the ***log_stats()*** method. These statistics are stored in a structured dictionary and are accessed by the User Interface for plotting and summary reporting.

Migrations and ***epidemics*** are regulated by cooldown intervals (***MIGRATION_COOLDOWN_DAYS***, ***EPIDEMIC_COOLDOWN_DAYS***) to avoid excessive intervention. When triggered, these events are applied uniformly across the world and involve random placement or removal of individuals according to defined parameters.

The ***GameManager*** includes a ***check_extinction()*** method that detects when one of the two animal species reaches zero population. If extinction occurs, the simulation is stopped.

In addition to real-time control, the class supports full ***serialization*** and ***deserialization*** of the simulation state through the ***save_state()*** and ***load_state()*** methods. These operations allow a user to preserve the current world and resume it at a later time, with all dynamic data and configuration values restored.

3.6 Main and Plotting

The final stage of the simulation architecture integrates the system's logic with the user-facing components responsible for configuration, execution and visualization. This layer is split across two modules:

- ***main.py***, which manages application startup, user configuration and game initialization.
- ***user_interface.py***, which defines ***PlanisussUI*** class, the graphical ***User Interface*** (UI), and real-time visual output using ***matplotlib***.

Together, these modules enable users to define initial conditions, control simulation execution and monitor ecosystem dynamics through interactive plots and controls. The following sections describe their respective responsibilities and design.

3.6.1 Main

The ***main.py*** module acts as the application's entry point and provides the interface through which users can configure simulation parameters and launch new or previously saved sessions. It encapsulates all logic related to user interaction prior to the start of the simulation and ensures that system-level components are initialized correctly and in accordance with user-defined values.

Upon launch, the program creates a launcher window using the ***tkinter*** library. This window offers two options: ***starting*** a new simulation or ***loading*** an existing one from a saved file, as shown in *Figure 1*.

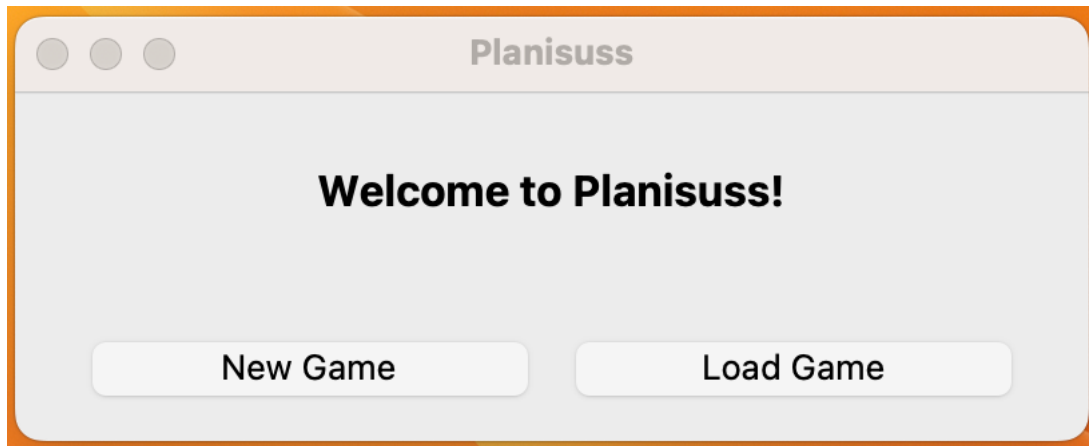


Figure 1: Welcome window.

If the user chooses to start a new simulation, the application opens a configuration window defined by the **GameConfigWindow** class, as shown in Figure 2. This interface exposes a subset of constants from **constants.py** that users can set directly. Each configurable parameter is accompanied by validation rules and input constraints to ensure correct formatting and logical consistency.

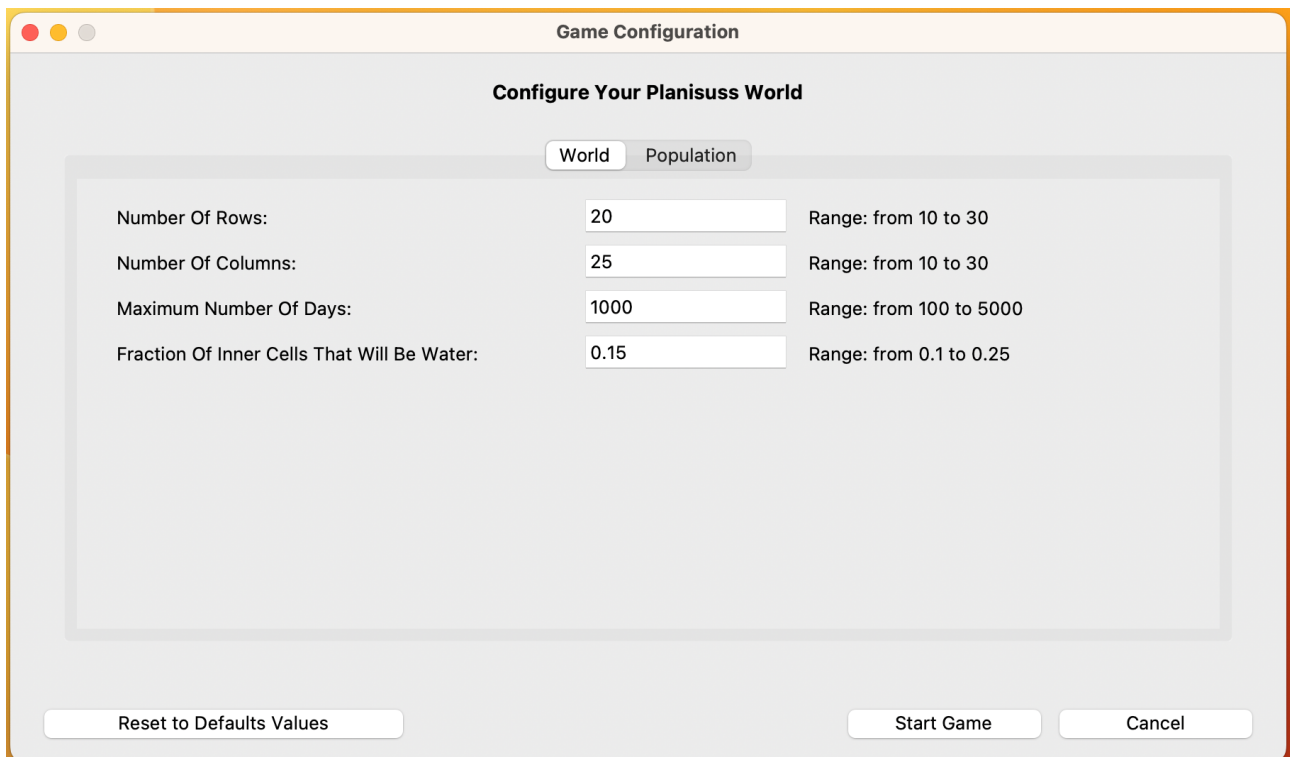


Figure 2: Game configuration window.

Configuration options are organized into tabbed sections using a **ttk.Notebook**, separating world parameters (such as grid size and water distribution) from population parameters (such as initial **Erbast** and **Carviz** ratios and group size limits). Each input field is linked to a specific constant and internally validated before the simulation can begin. Invalid values are rejected with informative error messages and users may also reset the form to default values.

Once validated, the user-defined values are applied to the **constants.py** module at runtime using reflection (**setattr**). This ensures that all modules importing constants will operate using the most recent configuration. After this step, the configuration window and launcher are closed, and a new

GameManager instance is created and initialized with the current settings. Control is then handed over to the **PlanisussUI** class, which manages the graphical interface and real-time visualization.

Alternatively, the user may choose to load a saved simulation by selecting a **.pkl** file from the file system. In this case, the application attempts to **deserialize** the simulation state using **GameManager.load_state()**. If successful, the saved game is restored, and the User Interface is launched with the previous state intact. Any **deserialization** errors are handled and reported through a dialog.

To summarize, the **main.py** module is designed for clarity and isolation of responsibilities. It does not directly interact with simulation internals beyond instantiating and passing the necessary objects. This separation allows the simulation logic and User Interface to evolve independently of the launch procedure, supporting both usability and maintainability.

3.6.2 User Interface (UI)

The **User Interface** (UI) for the **Planisuss** simulation is implemented in the **user_interface.py** module through the **PlanisussUI** class. This component is responsible for rendering real-time visual feedback, handling user controls and synchronizing the simulation state with the interface elements. The interface is built using **matplotlib**'s plotting and **widget** systems, with support from **tkinter** for auxiliary dialog boxes. An example of the UI is shown in Figure 3.

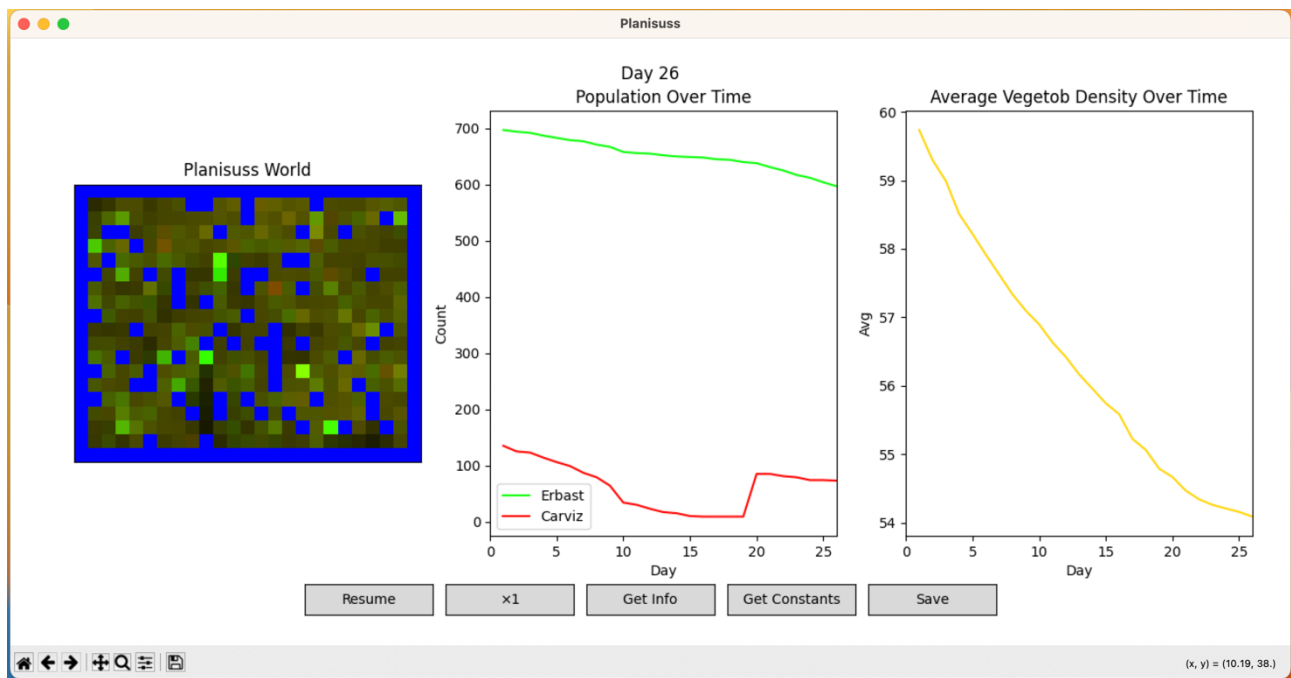


Figure 3: User Interface.

Upon instantiation, **PlanisussUI** receives an instance of the **GameManager**, which serves as the underlying simulation engine. If the world has not yet been initialized, the UI triggers its creation. The window layout is organized into three main plots:

- A **map view**, rendered as a color-coded image representing the current state of the grid. Species are mapped to **RGB** channels: **Carviz** in red, **Erbast** in green and **Vegetob** density contributing a yellow tint. **Water** cells are rendered in blue.

- A **population plot**, showing the number of **Erbasts** and **Carvizz** over time using line plots, updated at each simulation step.
- A **Vegetob density plot**, which tracks the average **Vegetob** density across the world as a function of time.

The UI also includes a set of interactive **control buttons** positioned below the plots. These allow the user to **pause** or **resume** the simulation, adjust the simulation **speed**, retrieve runtime **statistics**, display current **constants** and **save** the current state of the simulation. The buttons are implemented using **matplotlib.widgets.Button** and are connected to callback methods within the class.

The simulation is advanced using a timer-driven mechanism that repeatedly invokes the **tick()** method. This method steps the simulation forward by one or more **days** (depending on the current **speed multiplier**), updates the internal state and refreshes the visual elements accordingly. When the configured maximum number of **days (NUMDAYS)** is reached, the simulation is automatically paused, and the UI updates to indicate that the run has concluded.

User interaction is supported through several additional features:

- **Get Info Button** opens a dialog box summarizing current and initial statistics for both species and average **Vegetob** density, including population changes and social group counts.
- **Get Constants Button** displays all runtime-configurable constants and their current values.
- **Save Button** opens a file dialog to export the simulation state using the **save_state()** method of **GameManager**.
- **Window close handling** includes a confirmation dialog prompting the user to save the game before exiting.

The UI dynamically adjusts plot bounds, scales data and refreshes visuals to ensure legibility over long simulations. Plots are clipped to a moving window of recent days (**window_size**) to maintain clarity as the number of data points increases.

By isolating simulation logic from visualization and providing intuitive controls, the **PlanisussUI** class enables users to observe the consequences of parameter choices and emergent behaviour in the simulation in real time. Its modular construction and reliance on standard libraries ensure compatibility, extensibility and robustness across different usage contexts.

CHAPTER 4 – TESTING AND EVALUATION

This chapter presents the evaluation of the *Planisuss* simulation through a series of test runs configured under different parameter settings. The purpose of the evaluation is to observe system behaviour, verify the correctness of the model implementation and assess the qualitative and quantitative outcomes of the simulation under controlled conditions.

The analysis focuses on three aspects. First, the simulation scenarios and parameters are described, including the values used for initial populations, grid dimensions and time horizons. Next, the results and visualization outputs are presented, illustrating the evolution of species populations and *Vegetob* density through real-time plots and statistical summaries. Finally, performance observations are reported, with attention to computational efficiency, frame rate stability and simulation responsiveness across different scales.

4.1 Simulation Scenarios and Parameters

In order to verify the correctness, configurability and stability of the simulation system, multiple test runs were conducted using a variety of parameter sets. The tests were designed to assess not only the behavioural dynamics of the ecosystem but also the proper functioning of the configuration interface and enforcement of value constraints defined in *constants.py*.

Three principal scenarios were used to evaluate the range and behaviour of the system:

- A simulation using the **default values** preloaded in the configuration interface.
- A simulation using **minimum values** for all user-configurable parameters.
- A simulation using **maximum values** for all user-configurable parameters.

These tests were selected to verify the correct interpretation and application of boundary values as defined in the configuration UI. Screenshots of the configuration summary, retrieved using the **Get Constants button**, are included below to illustrate the parameters used in each case.

Across all scenarios, the simulations were executed up to the defined **NUMDAYS** horizon, unless early extinction occurred. The **default scenario** was used as a baseline to monitor typical system behaviour, while the **minimum** and **maximum scenarios** were used to stress-test the limits of grid size, population density and group capacity.

In total, a wide set of configurations was tested. Approximately all simulations run completed successfully, exhibiting consistent ecological behaviour and acceptable performance across different scales. In a small subset of cases, the simulation experienced computational slowdowns or instability, typically caused by **exponential population growth** of *Erbasts*. These edge cases will be addressed further in **sub-chapter 4.3 Performance Observations** and discussed in more detail in **Chapter 5 – Limitations and Future Work**.

The three scenarios described above provide representative coverage of the supported parameter space and confirm that the system responds appropriately to both conservative and extreme configurations.

The following three figures (i.e., *Figure 4*, *Figure 5* and *Figure 6*) show the parameter sets used in the **default**, **minimum** and **maximum value scenarios**, as retrieved from the configuration interface prior to execution.

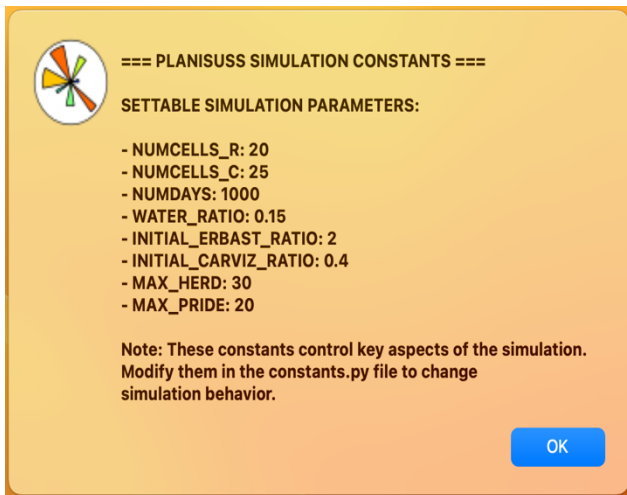


Figure 4: Default scenario.

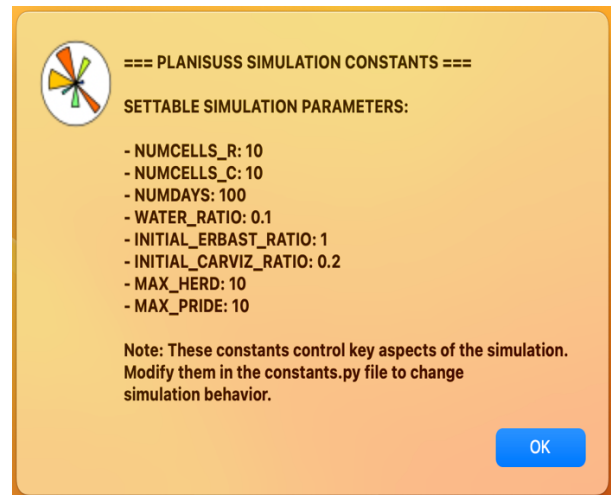


Figure 5: Minimum scenario.

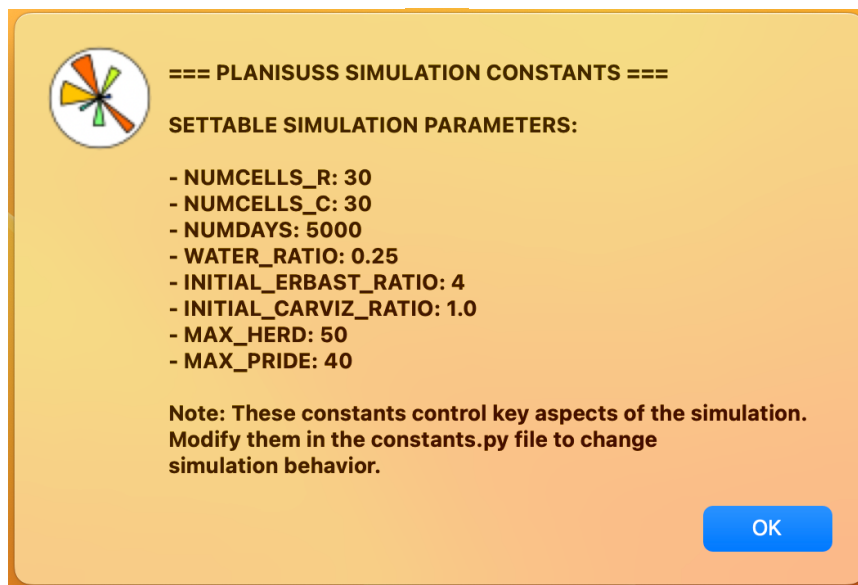


Figure 6: Maximum scenario.

4.2 Results and Visualization Output

The User Interface of the **Planisuss** simulation provides continuous visual feedback on the evolving state of the ecosystem. Each simulation run is displayed through three coordinated panels: a dynamic world map, a population trend plot and an average **Vegetob** density plot. These visual elements, combined with interactive status and summary messages, allow for effective monitoring and post-run evaluation.

To document the system's behaviour, several representative outputs have been collected. *Figure 7* captures a random moment during a simulation run, illustrating the active evolution of the world. The species distribution is rendered in real time, with colour intensities reflecting local population densities and **Vegetob** availability.

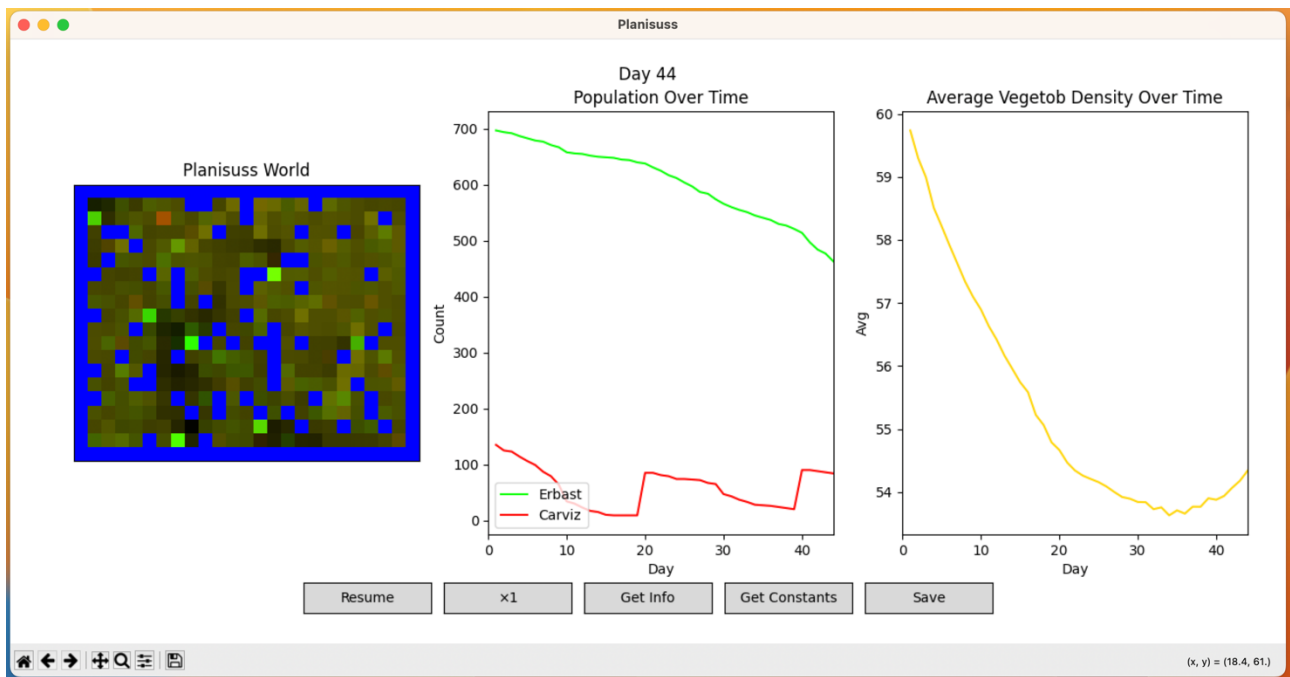


Figure 7: Random User Interface simulation.

Figure 8 shows the User Interface at the end of a simulation that reached **1000 days**, demonstrating the system's ability to maintain stability and render consistent results over long time horizons.

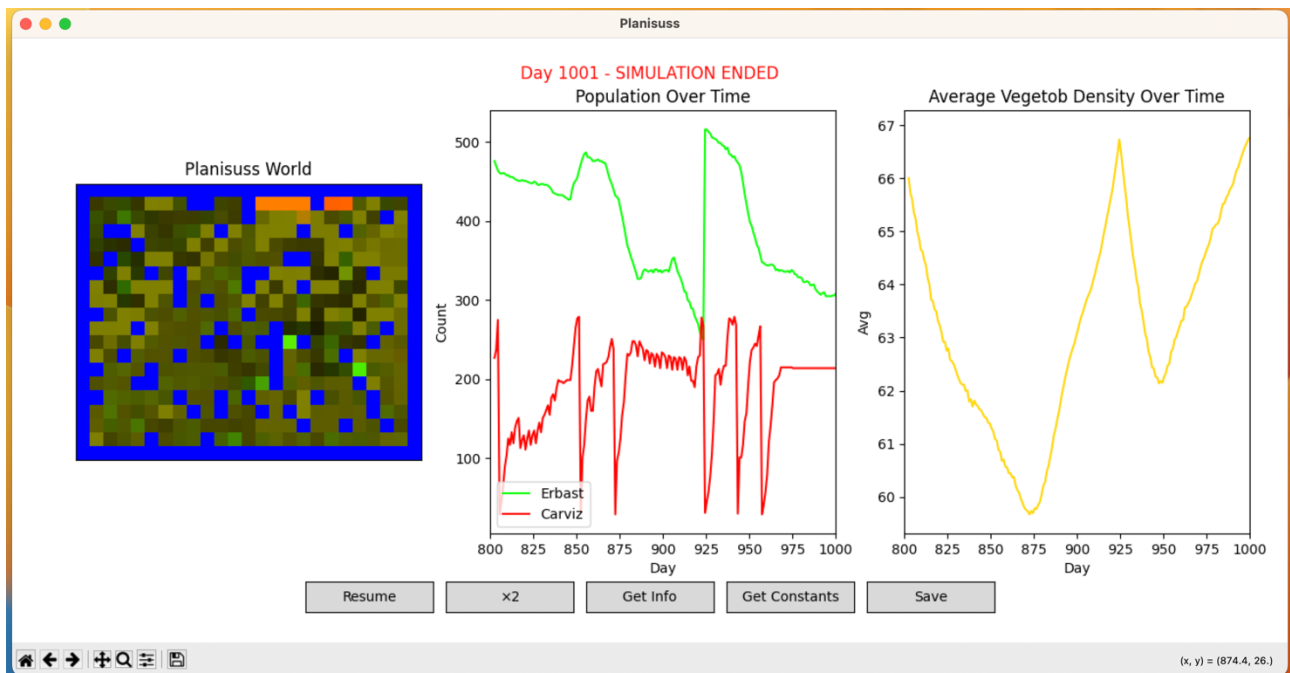


Figure 8: User Interface when simulation ended.

Additionally, Figure 9 captures a screenshot of the **Get Info** summary window for the case using default configuration values. This message box displays the initial and final statistics for **Erbasts**, **Carvizs**, average **Vegetob** density and social group counts. It confirms that all data series were correctly recorded and that the final population values remained within expected ranges.

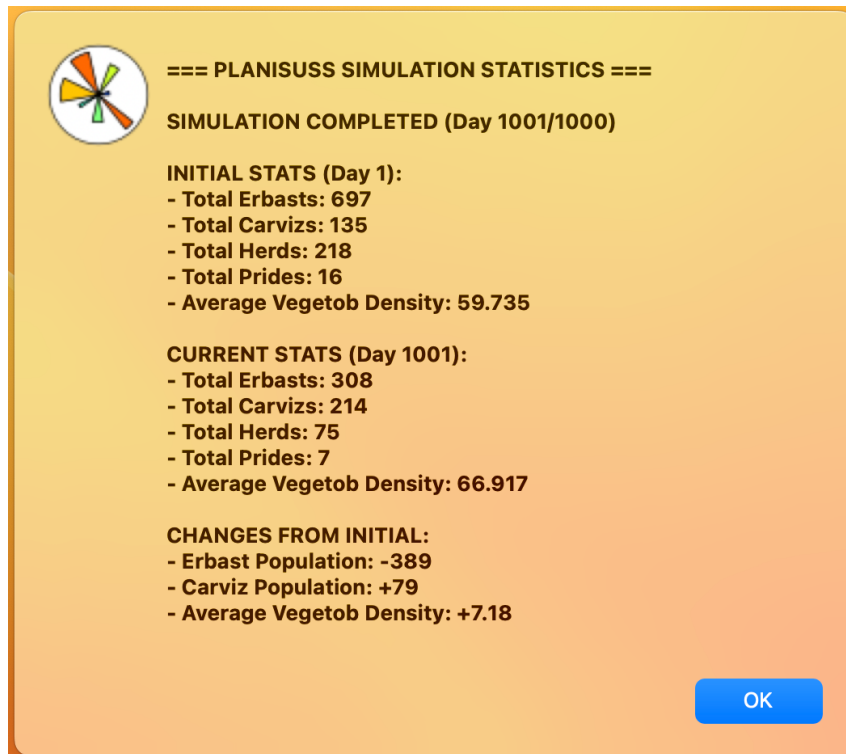


Figure 9: Get Info message at the end of default scenario.

Across all test runs, the simulation successfully captured realistic ecosystem dynamics. **Erbasts** and **Carvizz** demonstrated plausible population fluctuations, with periods of growth, decline and equilibrium. **Vegetob** regeneration and consumption cycles were visibly reflected in the density plots. In general, species were able to coexist over extended timeframes, with social groups forming, moving and dissolving as expected. While a few edge cases produced atypical behaviour due to extreme parameter combinations (discussed in the next sub-chapter), the overall results confirm the system's capacity to represent multi-species interactions in a spatially structured environment.

4.3 Performance Observations

From a performance standpoint, the simulation demonstrates stable and responsive behaviour under most configurations. The system is capable of iterating over all grid cells and updating multiple lists of animals and groups efficiently, even when the simulation spans several hundred or thousand days. During typical executions, especially when using default or moderate parameter values, the application maintains consistent frame rates and transitions between simulation phases occur without perceptible delay.

However, performance degradation was observed in certain edge cases, particularly in scenarios involving very high initial population densities combined with large grid dimensions. For instance, when the **INITIAL_ERBAST_RATIO** was set near the upper bound (four **Erbasts** per cell) and combined with a **30×30 cell grid**, the total number of **Erbasts** could grow rapidly and unpredictably. In some of these runs, a sharp population increase occurred after several hundred days, particularly around **day 700** or later, resulting in a notable slowdown in responsiveness or, in a few cases, termination of the program due to resource exhaustion.

These issues appear to stem primarily from the compounding complexity of population growth, which leads to increasingly expensive iterations over large numbers of individuals and social groups. Although such cases are rare, they highlight the computational sensitivity of the system to certain configurations. It is important to note that simulations using the same set of parameters may yield different outcomes: some complete successfully while others encounter slowdowns or crashes. This variation is due to the **stochastic** nature of the simulation, particularly the role of randomness in **Carviz** hunting attributes, which affects population control indirectly through hunt success or failure.

Throughout testing, adjustments were made to constants related to energy decay, reproduction constraints and group capacities in an attempt to mitigate runaway population growth. While these changes helped reduce the frequency of extreme cases, they did not eliminate the underlying behaviour, which is inherently linked to the interaction of multiple model components and random events.

In summary, the system performs reliably under typical use conditions and is capable of supporting long-term simulations with meaningful complexity. Performance issues are limited to parameter extremes and are primarily driven by user-defined inputs. These cases offer valuable insight into the limits of the current implementation and are further discussed in **Chapter 5 – Limitations and Future Work**.

CHAPTER 5 – LIMITATION AND FUTURE WORKS

While the *Planisuss* simulation achieves its primary objectives and supports a broad range of configurations, certain limitations emerged during development and testing. These are largely the result of trade-offs made between *realism*, *computational complexity* and *code maintainability*. This chapter outlines known issues, identifies areas for technical improvement and suggests possible directions for extending the simulation in future work.

5.1 Limitations, Improvements, Extensibility Considerations

One of the most significant limitations identified during testing concerns *computational scalability*. Although the system performs efficiently under most conditions, simulations configured with large grids and maximum population ratios may suffer from performance degradation or instability over long runtimes. In particular, scenarios involving high initial *Erbast* densities (e.g. four *Erbasts* per cell on a 30×30 grid) can lead to uncontrolled population growth after several hundred days. This behaviour results in increased iteration costs during simulation phases and, in some cases, memory exhaustion or program crashes.

The issue is made more complex by the *stochastic elements* of the simulation, especially those influencing predator-prey dynamics. Even when using identical parameters, two runs may produce diverging outcomes depending on how early hunting interactions unfold. In some cases, the *Carviz* population successfully regulates the growth of *Erbasts*; in others, early randomness allows the *Erbast* population to expand unchecked, overwhelming the system's processing capacity. Although several constants were tuned during testing to eliminate this effect (e.g. reproduction thresholds, energy decay), these adjustments could not fully eliminate the underlying issue without compromising the simulation's core mechanics.

Another limitation involves the absence of *defensive* programming in *getter* and *setter* methods. While *encapsulation* has been implemented consistently across classes, *input validation* and the use of *defensive copies* have been intentionally omitted to reduce overhead, as the system currently operates in a closed environment. However, this design choice limits the robustness of the code in scenarios where the system may later be integrated into external modules or exposed to untrusted input.

From a behavioural perspective, the movement logic and decision heuristics of species, while effective for demonstration purposes, are relatively simple. *Erbasts* and *Carvizes* rely on local desirability functions that do not adapt over time or incorporate long-term strategy. Group coordination, although implemented, is based on weighted averages and lacks mechanisms such as memory beyond recent cell visits or inter-group communication.

Despite these constraints, the system has been built with *modularity* and *extensibility* in mind, making it well suited for future development. Several enhancements and future works are possible, including:

- Improved *decision-making algorithms*, such as rule-based AI or reinforcement learning models for movement and hunting.
- Dynamic *environmental factors*, including climate variation, seasonal *Vegetob* growth patterns or habitat degradation.
- User-controlled interventions, enabling “*god-mode*” features such as spawning entities, triggering events or adjusting constants mid-simulation.

- **New species** or ecological roles, such as omnivores or decomposers, which could enrich trophic dynamics and introduce new inter-species interactions.
- **Parallelization** or **optimization** of simulation steps, especially for large grids, to improve performance in high-density scenarios.
- **Enhanced data logging** and replay capabilities, allowing deeper analysis of individual lifecycles, genealogies or emergent patterns.

Future work may also focus on integrating the simulation with analytical tools for automated evaluation of ecosystem metrics or for comparative analysis across parameter sets. These extensions would make the **Planisuss** system a more powerful platform for ecological modelling, educational demonstrations or experimentation with agent-based dynamics.

CHAPTER 6 – CONCLUSION

The development of the *Planisuss* simulation has provided a comprehensive opportunity to design, implement and evaluate a complex system grounded in algorithmic reasoning and structured programming. The project required modelling a multi-species ecosystem governed by energy dynamics, probabilistic interactions and spatial constraints.

Through this work, I was able to deepen my understanding of *algorithm design* and the effective use of *data structures*. From managing dynamic lists of animals and social groups, to implementing efficient neighbourhood queries and event triggers, the project required careful attention to computational logic, memory handling and temporal consistency. These challenges significantly strengthened my *programming skills*, both in terms of writing correct and reusable code and in structuring systems that scale with increasing complexity.

Beyond the technical aspects, the project highlighted the importance of balancing realism, performance and clarity. Decisions related to *encapsulation*, *parameter validation* and *event coordination* had to account for both functional correctness and long-term maintainability.

While certain limitations remain, particularly regarding *computational scalability* in extreme scenarios, the overall system has demonstrated robustness, flexibility and coherence across a wide range of test cases. It successfully models the emergence of complex dynamics from relatively simple behavioural rules.

This experience has not only reinforced my theoretical knowledge of simulation systems but also demonstrated the practical value of object-oriented design and algorithmic thinking.

TOOLS AND RESOURCES

Programming Language

- Python 3.12.4

Core Libraries

- NumPy 1.26.4 – Used for numerical structures and data manipulation.
- matplotlib 3.8.4 – Used for real-time plotting and User Interface rendering.

User Interface Framework

- tkinter – Used for configuration windows, dialogs and application control.

Save and load files

- pickle – Used to serialize and deserialize simulation state.

Development Environment

- Visual Studio Code.

Resources

- Wa-Tor simulation model.
- Conway's Game of Life.

REFERENCES

- K. Dewdney, “Computer Recreations: Sharks and fish wage an ecological war on the toroidal planet Wa-Tor,” *Scientific American*, vol. 251, pp. 14–22, December 1984.
- “Wa-Tor.” Wikipedia. <https://en.wikipedia.org/wiki/Wa-Tor>
- M. Gardner, “The Fantastic Combinations of John Conway’s New Solitaire Game 'Life',” *Scientific American*, vol. 223, pp. 120–123, October 1970.
- “Conway’s Game of Life.” Wikipedia. https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life
- matplotlib official website. <https://matplotlib.org/>
- NumPy official website. <https://numpy.org/>
- tkinter documentation. <https://docs.python.org/3/library/tkinter.html>
- S. Ferrari, “NumPy and Matplotlib Minimal Tutorial.” <https://elearning.unipv.it/mod/folder/view.php?id=67083>
- tkinter GeeksforGeeks tutorial. <https://www.geeksforgeeks.org/python/python-tkinter-tutorial/>
- “Constants for Planisuss.” https://elearning.unipv.it/pluginfile.php/269900/mod_folder/content/0/planisuss_constants.py
- Materials of the course.